

TRABALLO FIN DE GRAO
GRAO EN CIENCIA E ENXEÑARÍA DE DATOS

Visualizando el aprendizaje máquina: de la abstracción al entendimiento

Estudiante	Héctor Aldao Amoedo
Dirección:	Alejeandro Rodriguez Árias Samuel Magaz Romero

A Coruña, junio de 2026.

Dedicatoria¿?

Agradecimientos

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Etiam lobortis facilisis sem. Nullam nec mi et neque pharetra sollicitudin. Praesent imperdiet mi nec ante. Donec ullamcorper, felis non sodales commodo, lectus velit ultrices augue, a dignissim nibh lectus placerat pede. Vivamus nunc nunc, molestie ut, ultricies vel, semper in, velit. Ut porttitor. Praesent in sapien. Lorem ipsum dolor sit amet, consectetur adipiscing elit. Duis fringilla tristique neque. Sed interdum libero ut metus. Pellentesque placerat. Nam rutrum augue a leo. Morbi sed elit sit amet ante lobortis sollicitudin. Praesent blandit blandit mauris. Praesent lectus tellus, aliquet aliquam, luctus a, egestas a, turpis. Mauris lacinia lorem sit amet ipsum. Nunc quis urna dictum turpis accumsan semper. ¿?

Resumen

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Etiam lobortis facilisis sem. Nullam nec mi et neque pharetra sollicitudin. Praesent imperdiet mi nec ante. Donec ullamcorper, felis non sodales commodo, lectus velit ultrices augue, a dignissim nibh lectus placerat pede. Vivamus nunc nunc, molestie ut, ultricies vel, semper in, velit. Ut porttitor. Praesent in sapien. Lorem ipsum dolor sit amet, consectetur adipiscing elit. Duis fringilla tristique neque. Sed interdum libero ut metus. Pellentesque placerat. Nam rutrum augue a leo. Morbi sed elit sit amet ante lobortis sollicitudin. Praesent blandit blandit mauris. Praesent lectus tellus, aliquet aliquam, luctus a, egestas a, turpis. Mauris lacinia lorem sit amet ipsum. Nunc quis urna dictum turpis accumsan semper.

Abstract

Hello, here is some text without a meaning. This text should show what a printed text will look like at this place. If you read this text, you will get no information. Really? Is there no information? Is there a difference between this text and some nonsense like “Huardest gefburn”? Kjift – not at all! A blind text like this gives you information about the selected font, how the letters are written and an impression of the look. This text should contain all letters of the alphabet and it should be written in of the original language. There is no need for special content, but the length of words should match the language.

Palabras clave:

- First itemtext
- Second itemtext
- Last itemtext
- First itemtext
- Second itemtext
- Last itemtext
- First itemtext

Keywords:

- First itemtext
- Second itemtext
- Last itemtext
- First itemtext
- Second itemtext
- Last itemtext
- First itemtext

Índice general

1	Introducción	1
1.1	Contexto y motivación	1
1.1.1	Definición del Problema	1
1.1.2	Objetivos	2
1.1.3	Alcance y Limitaciones	2
1.1.4	Metodología de Trabajo	2
1.1.5	Estructura de la Memoria	2
2	Estado del Arte	3
3	Herramientas y técnicas	7
3.1	Godot como motor de desarrollo	7
3.2	Control de versiones y despliegue	10
4	Metodología y planificación	12
5	Desarrollo	13
5.1	Prototipo: versión 0.1	13
5.1.1	Primeras escenas	14
5.1.2	Exportación web con GitHub Pages	14
5.2	Árbol de decisión: versión 0.2	15
5.2.1	Primera arquitectura del árbol	15
5.2.2	Reorganización de la escena y primeros elementos explicativos	16
5.2.3	Animación de la construcción del árbol	19
5.2.4	Gráficas y casos explicativos en la ventana	21
5.2.5	Primer panel de selección de datos	24
5.2.6	Carga de archivos y navegación por pasos	25
5.2.7	Primera evaluación visual del árbol	26

5.3	Red neuronal: versión 0.3	28
5.3.1	Primera arquitectura visual de la red	28
5.3.2	Configuración de la arquitectura y representación lógica	28
5.3.3	Selección de datos y restricciones de la red	28
5.3.4	Algoritmo de entrenamiento incremental	28
5.3.5	Forward pass y visualización de datos	28
5.3.6	Cálculo del error y backward pass	28
5.3.7	Ventana informativa de la red neuronal	28
5.3.8	Inferencia con la red entrenada	28
6	Conclusiones	29
7	Trabajo futuro	30
A	Material adicional	32
	Bibliografía	35

Índice de figuras

2.1	Sección interactiva de la página web de 3Blue1Brown.	5
2.2	Página web TensorFlow Playground.	5
3.1	Entorno de desarrollo de Godot.	8
3.2	Diagrama general de la arquitectura de Godot.	9
5.1	Versión preliminar del menú principal.	14
5.2	Primera escena de prueba del árbol. Arriba a la derecha hay dos botones que sirven para guardar y cargar un árbol ya creado. Abajo a la izquierda se ve una sencilla estructura con forma de árbol, con los nodos rojos con un texto de ejemplo conectados por líneas negras. Sobre uno de los nodos, se ve el menú que permite añadirle un hijo o eliminarlo.	17
5.3	Primer menú de control del entrenamiento algorítmico, con los botones para iniciar el entrenamiento y avanzar al siguiente paso antes de traducir la interfaz y aplicar los temas visuales definitivos.	17
5.4	Panel inicial de la escena del árbol para escoger entre creación manual y creación algorítmica.	18
5.5	Interfaz de entrenamiento del árbol tras traducir los controles. A la izquierda se muestra el botón para comenzar el entrenamiento y a la derecha el estado del árbol al avanzar con el botón de siguiente paso, con temas distintos para nodos internos y hojas.	20
5.6	Primera versión de la ventana informativa integrada en la vista del árbol, todavía con un mensaje inicial que indica al usuario que debe comenzar el entrenamiento.	20
5.7	Entrenamiento del árbol en el que se aprecian los nodos pendientes.	23
5.8	Ventana informativa en la que se dibuja la gráfica de la entropía de cada atributo.	23
5.9	Menú que aparece tras seleccionar el modo algorítmico para escoger el conjunto de datos de entrenamiento.	24

5.10	Vista del árbol con el botón para retroceder al paso anterior.	27
5.11	Fase de evaluación en la que se ven dos datos ya clasificados y uno en proceso de clasificación.	27

Índice de cuadros

Capítulo 1

Introducción

PRIMER capítulo da memoria, onde xeralmente se exporán as liñas mestras do traballo, os obxectivos, etc. Incluimos un par de exemplos de citas e de referencias

LA inteligencia artificial ...?

1.1 Contexto y motivación

Los algoritmos de aprendizaje máquina son técnicas matemáticas que escapan de las fronteras de lo que es entendible a simple vista por ¿una persona cualquiera.¿ Desde sus raíces más puramente matemáticas hasta su implementación tanto hardware como software.

Esto ya era cierto en los tiempos de los primeros algoritmos de aprendizaje ¿por refuerzo, árboles o qué?, pero con el surgir de los modelos de aprendizaje profundo, esta realidad es más palpable que nunca.

Bajo estas premisas es que se siente indispensable ¿proporcionar a la gente los materiales y métodos para que puedan comprender lo mejor posible esta tecnología cada vez más omnipresente¿?. ¿Y a de más el aprendizaje es mejor si es interactivo ¿?

1.1.1 Definición del Problema

Hoy en día las principales formas de aprender cómo funcionan los modelos de inteligencia artificial (sobre todo los clásicos, pero también los modernos) es a través de los siempre confiables libros y universidad, las más ancestrales fuentes de conocimiento. Entre los formatos más actuales también encontramos los vídeos divulgativos en plataformas online como Youtube, donde la posibilidad de que todo el mundo comparta su contenido ha dado lugar a ¿muchas oportunidades de ver las interpretaciones visuales de otras personas¿?.

Sin embargo ¿aún no existe ninguna herramienta pedagógica que permita interactuar en tiempo real con los algoritmos.¿ ¿Programándolos se pueden ver sus resultados, su evolución en una gráfica, pero no palpar los que está sucediendo en las entrañas de la bestia mientras

mastica el equivalente a su comida: los datos, tanto en entrenamiento como en inferencia;?
((.....))

1.1.2 Objetivos

((Crear la aplicación - paso 1, hacerla - paso 2, exportarla
Publicarla para que todo el mundo la pueda disfrutar))
((creo qe hay q se rmás concreto))

Los objetivos que se plantean en este trabajo comienzan por plantearse qué algoritmos de IA son los más interesantes para ¿explicar¿. Esta es, en última instancia, una cuestión subjetiva, por lo que en este trabajo se abordó esta decisión mirando ¿de cara¿ a dos de los mayores paradigmas de la IA: la simbólica y el deeplearning. ((no se si decir de forma pragmática y decir que tenía en mente las redes neuronales, y que los árboles se parecían lo suficiente, o poner un poco de fantasía de "los árboles, uno los primeros amigos del hombre en cuánto a lo que hacer que una máquina aprenda a jugar al ajedrez..." (lo

De ambos se decidió que los mejores candidatos para protagonizar una explicación que busque asentar los fundamentos eran las versiones más fundamentales de cada uno. Del lado de la IA simbólica está el árbol de decisión ((planteado por bla bla bla en blabal)), conocido por ser ... white box Y en el deeplearning, el ladrillo fundacional, la red neuronal ((lo mismo de histoia...))

1.1.3 Alcance y Limitaciones

1.1.4 Metodología de Trabajo

1.1.5 Estructura de la Memoria

Estado del Arte

LA inteligencia artificial y el aprendizaje automático se apoyan en algoritmos cuyo funcionamiento interno no siempre resulta fácil de visualizar. Conceptos como el cálculo de una frontera de decisión o la actualización iterativa de los parámetros de un modelo pueden resultar abstractos cuando se estudian únicamente desde una formulación teórica. Esta dificultad se acentúa en el aprendizaje profundo, donde intervienen elementos como funciones de activación, pesos, sesgos, propagación hacia delante, retropropagación o gradientes. Por ello, se han desarrollado recursos educativos orientados a hacer más comprensible el comportamiento de estos modelos mediante explicaciones visuales y simulaciones interactivas. En el contexto de este trabajo, resultan especialmente relevantes las herramientas que permiten comprender el proceso de aprendizaje de los modelos, la relación entre sus componentes internos y la forma en la que una decisión algorítmica se transforma en un resultado observable.

Uno de los recursos divulgativos más conocidos para introducir las redes neuronales es la serie de vídeos *Neural networks* [1]. Esta fue publicada en el canal de YouTube *3Blue1Brown*. Este proyecto cuenta también con una página web [2], cuya principal aportación pedagógica es la posibilidad de interactuar con una red neuronal entrenada con el conjunto de datos MNIST [3]. La herramienta permite observar en tiempo real cómo la red reacciona a los cambios en el dato de entrada, representado mediante una superficie de dibujo en la que cada píxel se corresponde con una neurona de la capa de entrada, como se aprecia en la Figura 2.1.

Este tipo de recurso resulta valioso para desarrollar intuición, ya que reduce la barrera de entrada matemática y proporciona una primera aproximación visual al funcionamiento de una red. También es adecuado para motivar y contextualizar el problema gracias a una explicación apoyada en recursos visuales claros. Su formato, en cambio, conserva un carácter fundamentalmente expositivo: en la página web, dos de las diez lecciones incorporan un componente interactivo, y dicha interacción se orienta siempre a la relación entre la entrada y la salida de una red ya entrenada. El estudiante trabaja con una explicación limitada al conjunto MNIST y sin capacidad de experimentar de forma directa con la arquitectura del modelo. Esta

restricción deja abierta la necesidad de recursos más activos, en los que sea posible analizar el efecto de cada decisión de diseño sobre el comportamiento de una red neuronal.

Frente al formato audiovisual, existen herramientas que permiten manipular redes neuronales en tiempo real. Un ejemplo es *A Neural Network Playground* [4], una aplicación que permite configurar una red neuronal sencilla desde el navegador, seleccionar conjuntos de datos sintéticos, modificar hiperparámetros y observar cómo evoluciona la frontera de decisión durante el entrenamiento. En la Figura 2.2 se puede visualizar la página web.

A diferencia de los recursos puramente expositivos, la relevancia de TensorFlow Playground reside en que conecta elementos habitualmente tratados de forma abstracta con consecuencias observables. Modificar directamente la arquitectura del modelo (número de capas y neuronas) o sus hiperparámetros (tasa de aprendizaje y regularización) altera de manera inmediata el proceso de ajuste. Este enfoque interactivo facilita que el estudiante relacione conceptos críticos como el sobreajuste, la generalización o la convergencia con resultados palpables. La herramienta se centra principalmente en la frontera de decisión y en la pérdida, sin profundizar de forma detallada en operaciones internas como el cálculo de gradientes o la propagación de errores capa a capa.

Esta misma orientación hacia la experimentación aparece en herramientas centradas en arquitecturas más específicas. *GAN Lab* [5] permite experimentar en el navegador con redes generativas adversarias. Este tipo de modelo introduce una dinámica de aprendizaje distinta a la de una red supervisada clásica, ya que enfrenta dos redes: un generador, que intenta producir muestras plausibles, y un discriminador, que intenta distinguir entre datos reales y generados. La aportación principal de GAN Lab es representar visualmente una interacción compleja como la de un par de redes adversarias. La herramienta permite observar cómo cambian las distribuciones generadas durante el entrenamiento y cómo la competencia entre generador y discriminador afecta al resultado. Desde el punto de vista educativo, esto resulta útil para explicar que existen modelos de aprendizaje profundo con objetivos y esquemas de aprendizaje diferentes.

La limitación principal de *GAN Lab*, junto con la falta de detalle sobre ciertas operaciones internas, es que presupone una base de conocimiento mayor que la requerida para una introducción a las redes neuronales. Para comprender correctamente una GAN es necesario haber asimilado previamente ideas como función de pérdida, optimización, representación interna o entrenamiento iterativo. En consecuencia, *GAN Lab* es una herramienta valiosa para una fase posterior del aprendizaje, aunque no sustituye a una aplicación centrada en los fundamentos de una red neuronal básica.

En el ámbito de los algoritmos clásicos de aprendizaje automático, también existen propuestas para ilustrar el funcionamiento de los árboles de decisión. La herramienta web interactiva sobre árboles de decisión de Interactive Machine Learning [6] permite al usuario

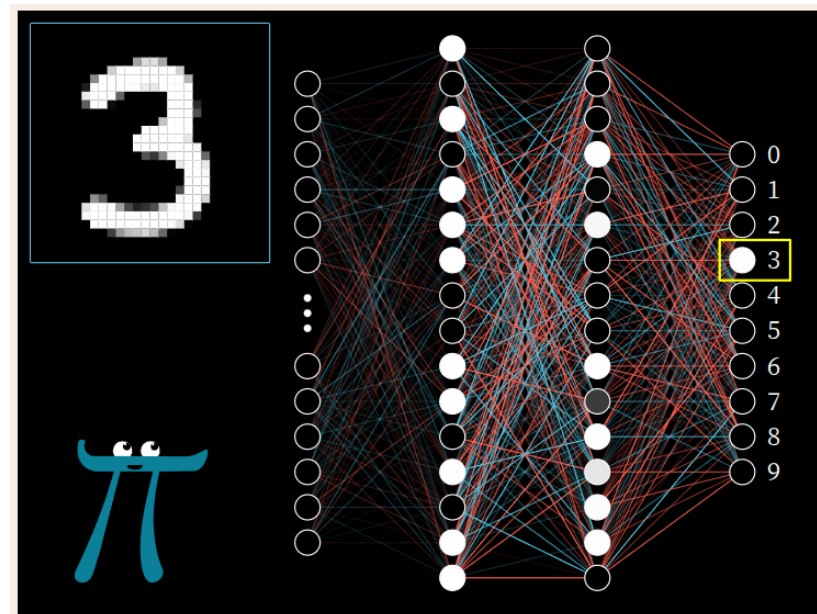


Figura 2.1: Sección interactiva de la página web de 3Blue1Brown.

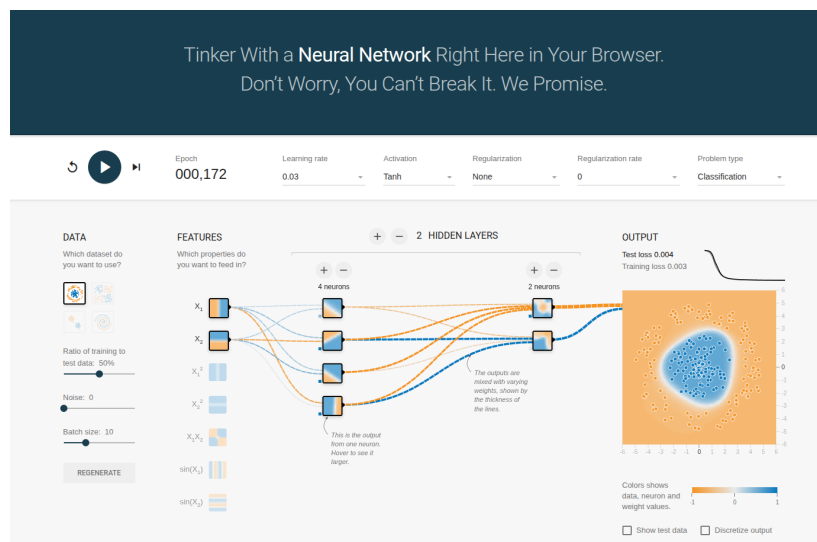


Figura 2.2: Página web TensorFlow Playground.

construir un árbol y observar el proceso de división de un conjunto de datos en tiempo real. Esta propuesta resulta interesante porque acerca al estudiante a la estructura jerárquica del modelo y al proceso de partición sucesiva de los datos. Su experiencia de uso, no obstante, es limitada y las explicaciones que acompañan a la simulación tienen poco desarrollo.

Aunque muestra los valores de distintas métricas, no acompaña dichos valores con una interpretación suficiente de su significado ni de su papel dentro del algoritmo. Esto reduce su utilidad como recurso formativo autónomo, especialmente para estudiantes que se aproximan por primera vez a los árboles de decisión.

En comparación con las redes neuronales, la disponibilidad de herramientas interactivas centradas en árboles de decisión es más reducida. La revisión realizada muestra una presencia mucho mayor de simuladores y visualizaciones dedicados a redes neuronales, tanto en recursos divulgativos como en herramientas web orientadas a la experimentación. Esta diferencia es especialmente visible cuando se buscan recursos que combinen visualización del modelo, explicación del criterio de partición y evaluación paso a paso.

Desde la perspectiva de este proyecto, la principal oportunidad se encuentra en combinar las fortalezas de estos recursos. Una herramienta educativa eficaz debería mantener la claridad visual de los recursos divulgativos y su atención a la base matemática, incorporar la manipulación directa de los simuladores y ofrecer suficiente rigor técnico para que lo presentado al usuario se corresponda con el funcionamiento real del algoritmo. También debería evitar que la interacción se limite a modificar hiperparámetros globales. Para entender una red neuronal resulta especialmente útil poder observar el flujo de datos, la contribución de los pesos y el efecto del entrenamiento sobre cada componente del modelo. En el caso de los árboles de decisión, la visualización debería mostrar cómo se selecciona cada atributo, cómo se divide el conjunto de datos y cómo se alcanza una clasificación final a partir de las características de entrada. En ambos modelos, esta visualización debe ir acompañada de explicaciones textuales que interpreten las métricas, los cálculos y las decisiones del algoritmo, de forma que la simulación no dependa únicamente de la observación visual.

Herramientas y técnicas

EL desarrollo de este trabajo se apoya en un conjunto de herramientas orientadas a construir una aplicación educativa, interactiva y ejecutable desde el navegador. La elección tecnológica del entorno de desarrollo tuvo en cuenta tanto criterios de implementación como la naturaleza del problema abordado: explicar modelos de aprendizaje automático mediante visualizaciones dinámicas, animaciones y ejemplos manipulables por el usuario. En consecuencia, se priorizaron tecnologías que facilitasen la construcción de interfaces interactivas, la validación de los algoritmos implementados y la publicación del resultado final en un entorno accesible. Godot permite exportar un mismo proyecto a distintas plataformas, como escritorio, móvil o web. En este caso, la versión web se escogió por su accesibilidad: el usuario puede abrir la aplicación desde un navegador, sin instalar software ni preparar un entorno de ejecución específico.

3.1 Godot como motor de desarrollo

La herramienta principal empleada fue Godot, un motor de desarrollo libre y multiplataforma orientado a la creación de aplicaciones interactivas [7]. Sus características resultan adecuadas para una aplicación educativa como la desarrollada en este trabajo. El proyecto requiere presentar información visual, responder a acciones del usuario, animar procesos paso a paso y mantener una estructura clara entre los distintos elementos de la interfaz. Estas necesidades encajan de forma natural con el modelo de escenas, nodos y señales que propone Godot [8].

No se si este párrafo anterior enlaza bien con este siguiente

Y si podeis porfavor revisar que se entienda bien la explicación de los nodos.

A lo largo de esta memoria se hablará en repetidas ocasiones de dos de las principales estructuras con las que se trabaja en Godot: las escenas y los nodos.

Los nodos son el elemento básico y más importante con el que se trabaja en el motor. Todo

elemento y subelemento que vaya a estar presente en tiempo de ejecución en el programa es un nodo. Y cada nodo cuenta, principalmente, con una serie de atributos, métodos y señales. Igual que en la programación orientada a objetos, los atributos son variables o constantes y los métodos funciones. Las señales son disparadores de funciones. Cuando son emitidas llaman a la función a la que están conectadas. Son una forma de conectar eventos entre diferentes nodos. Son el núcleo del paradigma de programación que promueve el motor: la programación dirigida por eventos.

Los atributos, métodos y señales de cada nodo vienen definidos por su clase. Godot dispone de un gran árbol de clases predefinidas que heredan atributos y métodos de sus clases padre. Las tres clases básicas son: el *Node* una clase vacía, la raíz del resto de nodos, que solo cuenta con los métodos mínimos para que funcione en el motor; el *Node2D* característico por contar con atributos referentes a la posición en un espacio bidimensional; el *Node3D* similar al anterior pero en un espacio tridimensional; y el *Control* pensado para los elementos de interfaz de usuario. Por encima de estas, el desarrollador siempre puede definir sus propias clases ampliando las ya existentes.

Los nodos por sí solos son clases abstractas que no existen en el programa ejecutable final por sí mismos. Para instanciarlos están las escenas. Estas son la materialización concreta del proyecto: los archivos que son guardados en disco y con los que se interactúa en el editor como se puede ver en la Figura 3.1. Están formadas, como mínimo, por un nodo raíz. De este nodo raíz pueden colgar nodos hijo, que a su vez pueden contar con sus propios nodos hijo, y así sucesivamente.

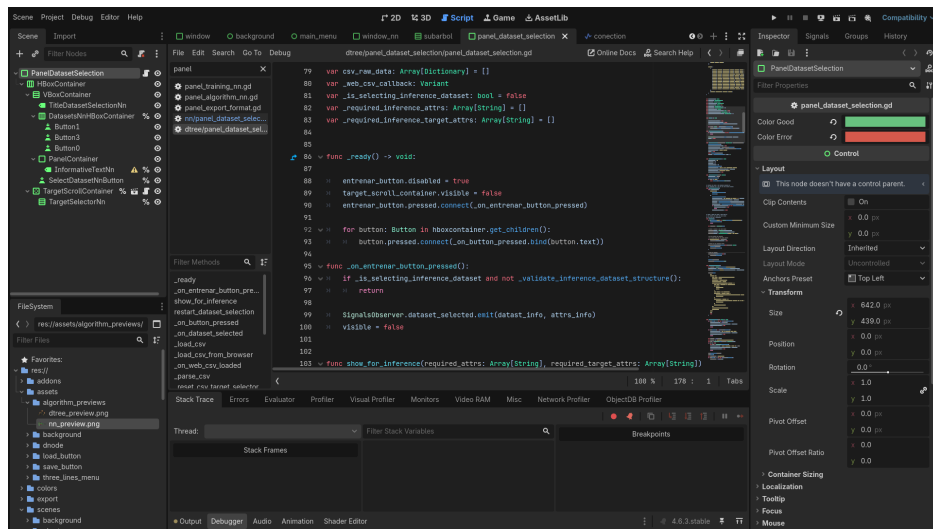


Figura 3.1: Entorno de desarrollo de Godot.

Hace falta que en el pie de foto de 3.1 describa en mayor profundidad lo que hay en pantalla?

Finalmente, las escenas, como en última instancia son árboles de nodos, pueden ser ins-

tanciadas dentro de otras escenas. Esto habilita una modularidad total, puesto que toda estructura de nodos que sea susceptible de ser reutilizada, puede guardarse como una escena para no tener que ser recreada.

Además, los nodos pueden tener asociado un script. Estos son código desde el que se puede interactuar, principalmente, con los atributos y métodos del nodo al que están asociados, así como con sus hijos. Pero no están limitados a estos elementos. Siempre se puede acceder al nodo raíz y, desde este, modificar cualquier propiedad global o acceder a cualquier nodo en la escena. Además, mediante la creación de *Autoloads* en el proyecto también se puede acceder a clases personalizadas en cualquier punto del código.

La Figura 3.2 muestra de forma esquemática esta organización y sitúa las escenas, los nodos, los scripts y los módulos dentro de la arquitectura general del motor.

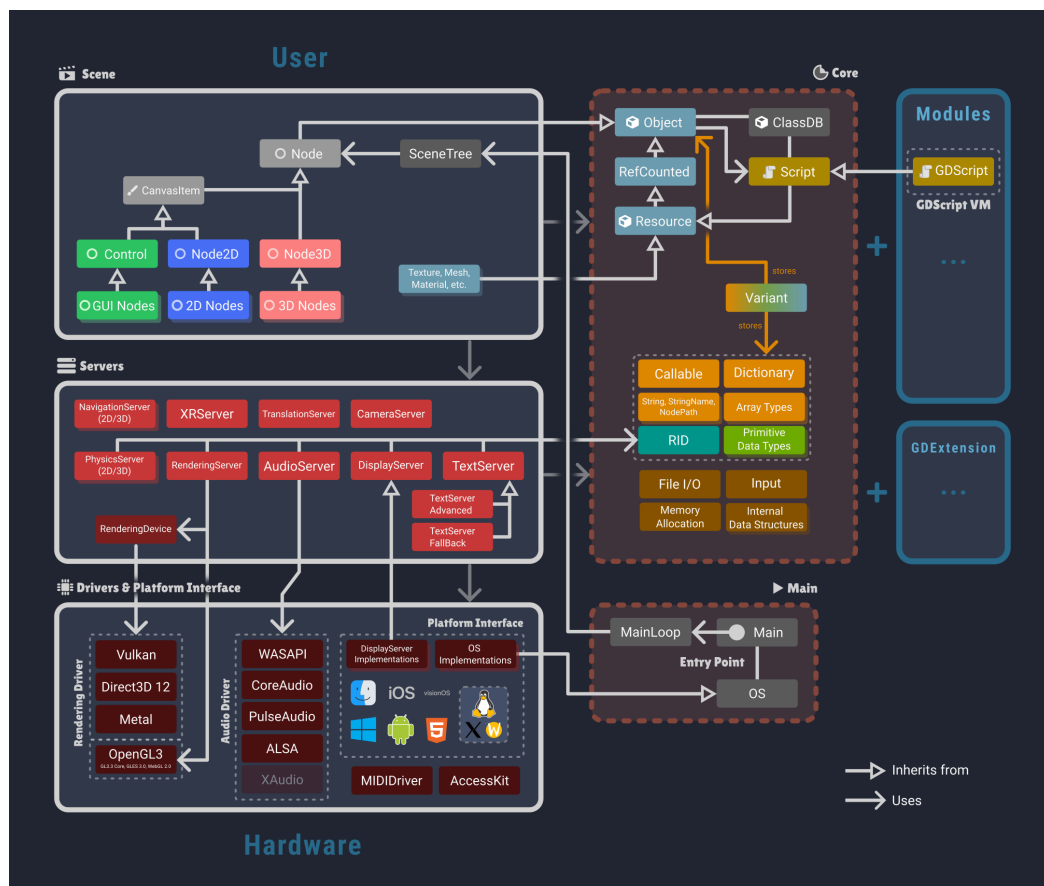


Figura 3.2: Diagrama general de la arquitectura de Godot.

La organización en escenas permite separar componentes de la aplicación con responsabilidades distintas, como las vistas de entrenamiento, los paneles explicativos, las representaciones gráficas y los controles de interacción. Este enfoque favorece una estructura modular en la que cada parte de la interfaz puede desarrollarse, probarse y reutilizarse con menor aco-

plamiento. El sistema de señales facilita la comunicación entre elementos de la aplicación sin imponer dependencias directas innecesarias, lo que resulta útil en un entorno en el que las acciones del usuario deben provocar cambios visuales y actualizaciones del estado interno de los modelos.

La manera en que Godot organiza las interfaces mediante nodos permite estructurar los elementos de los algoritmos tratados de forma sencilla. Un árbol de decisión puede mostrarse como una estructura jerárquica de nodos conectados, mientras que una red neuronal puede representarse mediante capas, neuronas y conexiones. Esta correspondencia conceptual ayuda a trasladar las estructuras internas del algoritmo a elementos visuales manipulables dentro de la aplicación.

Otro motivo importante para escoger Godot fue su sistema de exportación multiplataforma. El motor permite generar versiones para diferentes entornos a partir del mismo proyecto, incluida una versión HTML5 de la aplicación [9]. La exportación web se adoptó por la forma en la que se quería distribuir la herramienta: una aplicación educativa debe ser sencilla de abrir, compartir y probar. Publicarla como aplicación web evita exigir al usuario una instalación local y facilita el acceso desde un navegador. Esta característica es especialmente relevante en un proyecto con finalidad educativa, ya que reduce la fricción de uso y permite distribuir la herramienta como una aplicación web estática.

No tengo claro que ya se estén dando suficientes argumentos a favor de usar godot

3.2 Control de versiones y despliegue

Para la gestión del código fuente se utilizó Git, un sistema de control de versiones distribuido ampliamente utilizado [10]. Su uso permitió mantener un registro de los cambios y trabajar de forma incremental. Esto resultó útil durante el desarrollo, ya que la aplicación fue creciendo mediante pruebas sucesivas: primero se validó la interacción básica, después se incorporaron los algoritmos y más adelante se añadieron explicaciones y mejoras de interfaz. Disponer de un historial de cambios permitió avanzar con mayor seguridad y conservar puntos estables del proyecto mientras se realizaban modificaciones.

El repositorio se alojó en GitHub, lo que proporcionó almacenamiento remoto y una copia accesible del proyecto. Además, se utilizó GitHub Pages para publicar la versión web de la aplicación [11]. Este servicio permite desplegar sitios estáticos directamente desde un repositorio, por lo que encaja con la exportación web generada por Godot. La aplicación exportada se publica como un conjunto de archivos preparados para ejecutarse en el navegador, distinto del proyecto editable de Godot.

Esta fase tiene importancia porque marca la diferencia entre una aplicación que funciona dentro del entorno de desarrollo y una herramienta que puede utilizar cualquier persona desde

un enlace. En el editor, el proyecto se ejecuta en un contexto controlado, con acceso directo a las escenas, scripts y recursos locales. En el navegador, la aplicación depende de los archivos exportados, de las restricciones propias de la plataforma web y de la forma en la que el servicio de publicación sirve esos archivos. Por este motivo, el despliegue se trató como una parte más del desarrollo y no únicamente como el último paso administrativo.

La combinación de Godot y GitHub Pages permitió que el resultado final pudiese consultarse desde el navegador sin infraestructura de servidor adicional. Esto encaja con el objetivo de accesibilidad planteado al escoger la exportación web: el usuario solo necesita abrir una dirección, mientras que el proyecto mantiene un proceso de publicación suficientemente simple para ser actualizado durante el desarrollo.

Metodología y planificación

Capítulo 5

Desarrollo

En este capítulo se describe el proceso seguido para construir la aplicación. La explicación se plantea como una evolución por versiones. Comienza por comprobar que la idea es viable. Después se desarrolla el árbol de decisión, desde sus inicios hasta el resultado funcional con todas las características que se buscaban. Continúa con el desarrollo de la red neuronal, pasando por todas sus fases de creación y mejora. Y termina con el pulido final de la interfaz y retoques a todas las características con las que cuenta el resultado final.

aquí "característica" lo estoy traduciendo de "feature", creo q puede estar bien, me genera dudas porque me está sonando algo raro

5.1 Prototipo: versión 0.1

El primer objetivo consiste en comprobar que la herramienta elegida sirve para crear una aplicación interactiva y publicarla como página web. Antes de invertir más tiempo en el desarrollo, es necesario verificar tres aspectos básicos: que Godot permite crear una interfaz sencilla, que se puede representar una estructura con forma de árbol y que el resultado puede ejecutarse desde el navegador.

Por ese motivo, la primera versión del proyecto es deliberadamente simple. Consiste en un menú inicial y en una escena donde el usuario puede crear pequeños elementos conectados entre sí. Al pulsar sobre uno de ellos aparece una opción para añadir un nuevo elemento debajo, de manera que poco a poco se forma una estructura parecida a un árbol. aquí asumo que cuando hablo de árbol se entiende que es la estructura de datos (por lo tanto que se ve "del revés" a como se ve un árbol real) y no un árbol real, no?? Esta prueba está todavía lejos de un árbol de decisión real. Su valor está en comprobar si la interacción principal de la aplicación tiene sentido.

5.1.1 Primeras escenas

La primera escena creada es el menú principal. En ese momento tiene un único botón, ya que su papel es servir como punto de entrada hacia la prueba del árbol. Incluir este menú desde el principio ayuda a pensar la aplicación como un conjunto de módulos y facilita su ampliación posterior. En la Figura 5.1 se ve esta primera versión.

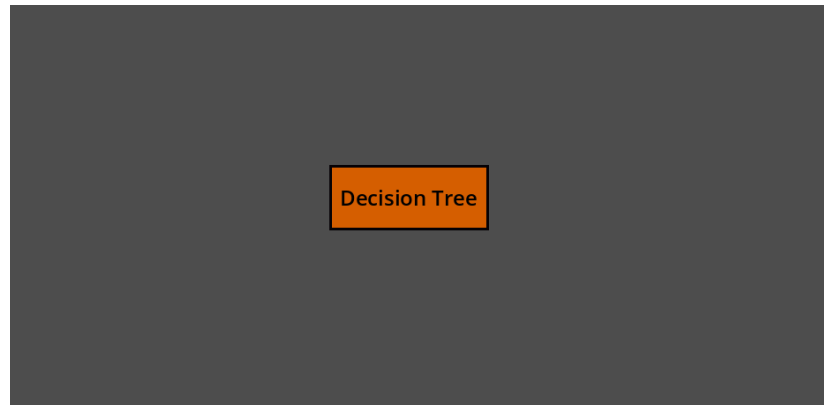


Figura 5.1: Versión preliminar del menú principal.

Al entrar en la escena del árbol aparece un único elemento inicial. Desde él se pueden crear nuevos elementos hijos, conectados por líneas y colocados bajo el anterior. También se añade la posibilidad de borrar partes del árbol, con el fin de comprobar que la estructura puede cambiar durante la ejecución sin que la escena deje de ser comprensible.

Esta primera prueba tiene una finalidad muy concreta: validar la interacción. Lo importante es comprobar que el usuario puede construir una jerarquía visual, que las conexiones se actualizan y que la pantalla sigue siendo manejable al crecer el número de elementos. En la Figura 5.2 se muestra cómo era esta escena inicial.

También se dejan visibles dos botones relacionados con cargar y guardar. En esta versión todavía no tienen una utilidad real. Su presencia sirve para reservar un espacio de la interfaz a una necesidad que ya se prevé: poder conservar un árbol y recuperarlo más adelante.

5.1.2 Exportación web con GitHub Pages

Una vez que la prueba interactiva funciona dentro de Godot, el siguiente paso lógico es llevarla al navegador. Esta comprobación es esencial, ya que la aplicación está pensada para ser consultada como una página web y también debe poder probarse fuera del editor de desarrollo.

La exportación se realiza correctamente con este proyecto inicial. El resultado se publica mediante GitHub Pages, usando la configuración del propio repositorio para indicar qué carpeta debe servir como página web. Con esto se confirma que el flujo completo es viable:

desarrollar en Godot, exportar el proyecto y publicarlo en una dirección accesible desde el navegador.

Esta prueba temprana reduce el riesgo del proyecto. Si la exportación web da problemas graves, hace falta replantear la tecnología antes de avanzar con los algoritmos. Al funcionar desde el inicio, el desarrollo puede continuar sobre una base más segura.

Añado aquí también algo de chicha sobre la exportación al ghpages?

5.2 Árbol de decisión: versión 0.2

El proyecto deja de ser una prueba de interacción cuando comienza el desarrollo del primer algoritmo. A partir de esta versión, el objetivo pasa de dibujar elementos conectados a construir un árbol de decisión real, mostrar su evolución paso a paso y explicar las decisiones tomadas durante el entrenamiento. La dificultad principal está en coordinar tres niveles: el algoritmo que calcula la siguiente división, la estructura lógica del árbol y la escena que el usuario ve en pantalla.

5.2.1 Primera arquitectura del árbol

El primer cambio consiste en sustituir la escena de prueba por una vista preparada para un árbol construido por algoritmo. La prueba inicial servía para validar que se podían añadir y borrar nodos manualmente, aunque su estructura resultaba insuficiente para un entrenamiento real. Un árbol de decisión puede crecer varios niveles y abrir muchas ramas en poco espacio, por lo que una pantalla fija deja de ser útil en cuanto el número de nodos aumenta. Por esta razón se incorpora una zona desplazable dentro de la interfaz, de manera que el usuario pueda moverse por el árbol cuando su tamaño supere el área visible.

También se revisa la distribución de los nodos. En la versión manual bastaba con colocar cada hijo bajo su padre, ya que el objetivo era comprobar la interacción. Durante el entrenamiento algorítmico se necesita una disposición más estable: los hijos deben aparecer por debajo, el padre debe quedar centrado respecto a ellos y el árbol debe reorganizarse cada vez que se añade una rama. Para conseguirlo se introduce un cálculo de posiciones basado en la profundidad de cada nodo y en su orden dentro de la estructura. Así se evita que las ramas se acumulen en la misma zona y se mantiene una lectura jerárquica clara.

Otra decisión importante es separar la información lógica del árbol de su representación visual. Cada nodo visible conserva un identificador, una referencia al padre, una lista de hijos, su profundidad y la información que representa, como el atributo de decisión o la etiqueta final. Al mismo tiempo, el árbol mantiene un diccionario interno que permite localizar cada nodo por su identificador. Esta duplicidad resulta útil porque la escena de Godot sirve para dibujar, animar y recibir eventos, mientras que el diccionario permite recorrer la estructura,

recalcular posiciones, eliminar subárboles y evaluar ejemplos más adelante.

Las conexiones entre nodos se tratan como elementos propios. Cada conexión representa una rama del árbol y puede mostrar el valor del atributo que conduce desde el nodo padre hasta el hijo. Esta separación permite que los nodos se encarguen de mostrar decisiones o clasificaciones, y que las conexiones expliquen el camino seguido. Más adelante, esta decisión facilita tanto la animación de las ramas como la evaluación visual de nuevos datos.

Por ejemplo, en esta frase de aquí yo creo que el presente continuo queda mal, y que habría que poner más bien "se añadió el modo automático" Con la base visual preparada, se añade el modo automático. En este modo, el usuario controla el avance del entrenamiento en lugar de añadir ramas manualmente. La primera versión de la interfaz es sencilla, con un botón para iniciar el proceso y otro para avanzar al siguiente paso, como se muestra en la Figura 5.3. Los textos aparecen todavía en inglés porque pertenecen a una fase temprana de la aplicación.

La construcción paso a paso se apoya en una separación de responsabilidades. El algoritmo calcula qué tipo de nodo debe crearse en cada momento y emite la información necesaria para hacerlo: padre, valor de la rama, atributo elegido, etiqueta si se trata de una hoja y datos explicativos del paso. El controlador recibe esa información y la traduce a cambios en la escena, creando o actualizando los nodos visibles. De este modo, la lógica de aprendizaje queda aislada de los elementos gráficos de Godot, y la vista puede centrarse en representar el resultado de cada decisión.

Para que el usuario pueda avanzar con el botón de siguiente paso, el algoritmo no construye todo el árbol de una sola vez. El trabajo pendiente se guarda como una serie de contextos, cada uno con los datos que llegan a una rama, los atributos disponibles, el padre visual y la profundidad correspondiente. Cada pulsación extrae uno de esos contextos, decide si debe convertirse en una hoja o en un nodo interno, actualiza la escena y deja preparados los contextos que quedan por procesar. Esta organización incremental ayuda a que el árbol pueda explicarse nodo a nodo sin necesidad de tener que almacenar toda la información a la que se quiere acceder en memoria.

Las primeras pruebas se realizan con un conjunto de datos pequeño y controlado. Su finalidad es comprobar que el algoritmo crea la raíz, abre ramas, coloca los nodos en posiciones coherentes y mantiene sincronizados el estado interno y la vista. En esta fase queda fijado el patrón principal del resto del desarrollo del árbol: el algoritmo decide, el controlador traduce esa decisión y el árbol visible se reorganiza tras cada paso.

5.2.2 Reorganización de la escena y primeros elementos explicativos

Cuando el modo automático empieza a funcionar, la escena se reorganiza para reducir elementos provisionales. La vista del árbol pasa a integrarse mejor con los contenedores de interfaz de Godot, lo que permite combinar botones, paneles y zonas desplazables de forma

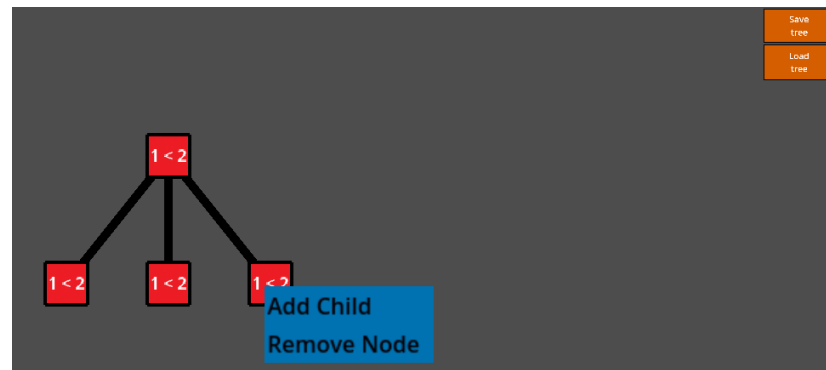


Figura 5.2: Primera escena de prueba del árbol. Arriba a la derecha hay dos botones que sirven para guardar y cargar un árbol ya creado. Abajo a la izquierda se ve una sencilla estructura con forma de árbol, con los nodos rojos con un texto de ejemplo conectados por líneas negras. Sobre uno de los nodos, se ve el menú que permite añadirle un hijo o eliminarlo.

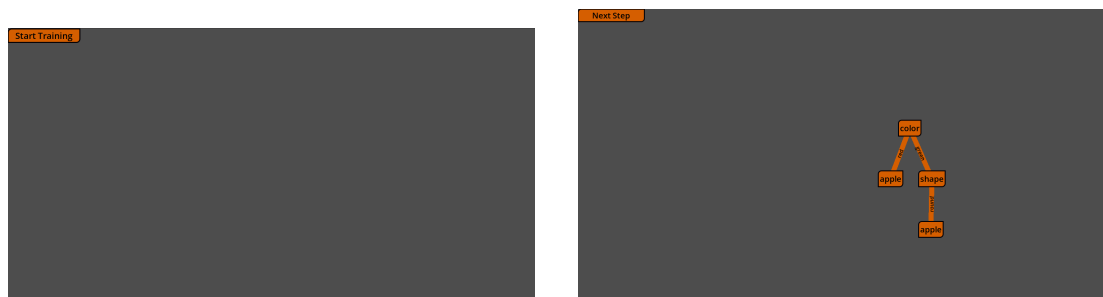


Figura 5.3: Primer menú de control del entrenamiento algorítmico, con los botones para iniciar el entrenamiento y avanzar al siguiente paso antes de traducir la interfaz y aplicar los temas visuales definitivos.

más natural. Esta reorganización también mejora el centrado del árbol. En lugar de mover la escena completa, se recalculan las posiciones internas de los nodos y el área mínima del contenedor desplazable. Así el árbol aparece centrado cuando cabe en pantalla y sigue pudiendo recorrerse cuando crece.

La escena se convierte entonces en un flujo guiado. Al entrar en la vista del árbol se muestra primero un panel que permite escoger entre creación manual y creación algorítmica. Cada opción activa controles distintos y evita mezclar acciones pensadas para usos diferentes. En la Figura 5.4 se muestra este panel inicial.

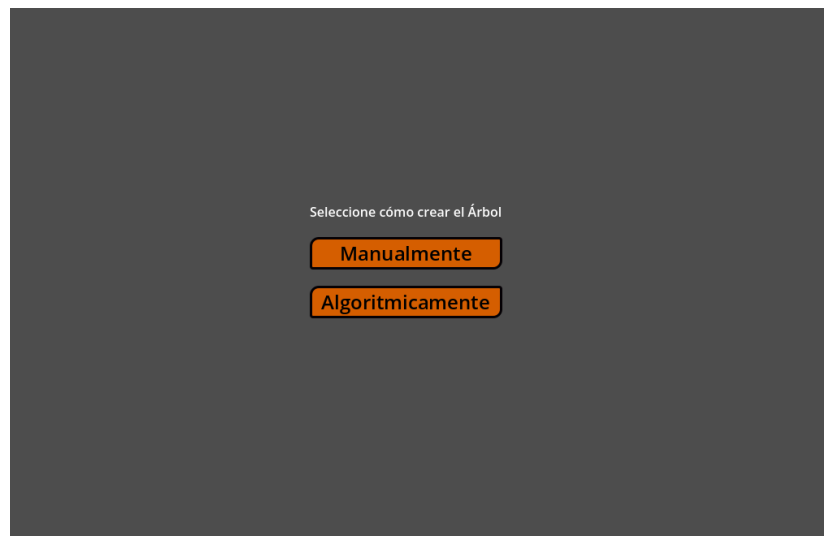


Figura 5.4: Panel inicial de la escena del árbol para escoger entre creación manual y creación algorítmica.

En esta etapa también se mejora la apariencia de los nodos. Los nodos internos, que representan preguntas sobre un atributo, y las hojas, que representan una clasificación final, reciben estilos visuales distintos. Esta distinción permite interpretar la estructura con mayor rapidez: algunos nodos indican que todavía hay una decisión que tomar, y otros señalan que se ha llegado a un resultado. Los nodos creados por el algoritmo quedan protegidos frente al menú de edición manual, ya que representan el resultado del entrenamiento y deben cambiar solo cuando avanza o retrocede el proceso.

La interfaz se traduce al castellano en este mismo tramo. Los controles principales pasan a llamarse “Empezar entrenamiento” y “Siguiendo paso”. El cambio ayuda a que la vista del árbol sea coherente con el idioma del resto de la aplicación y con el tono explicativo del trabajo. En la Figura 5.5 se ve la interfaz tras esta adaptación y los primeros cambios visuales.

Una vez que la construcción visual resulta estable, se incorpora la ventana informativa. Hasta este punto, la escena permite ver qué se añade al árbol, aunque todavía ofrece poca información sobre el motivo de cada decisión. La ventana nace para acompañar cada paso

con una explicación escrita y sincronizada con el estado interno del algoritmo. Su primera versión contiene un mensaje inicial que invita a comenzar el entrenamiento, como se observa en la Figura 5.6.

La ventana se implementa como un componente separado de la vista principal. Esta separación permite actualizar su contenido cada vez que el algoritmo procesa un contexto, sin mezclar la redacción de explicaciones con la creación de nodos en pantalla. El algoritmo prepara un conjunto de datos explicativos del paso actual: etiquetas presentes, número de ejemplos, atributos disponibles, tipo de nodo creado, atributo escogido y valores de la métrica usada para comparar alternativas. La ventana recibe esa información y la convierte en texto dirigido al usuario.

Los primeros textos cubren tres casos fundamentales. El primero aparece cuando se crea un nodo interno y hay que escoger por qué atributo dividir los datos. El segundo se usa cuando todos los ejemplos que llegan a una rama comparten la misma etiqueta y se puede crear una hoja pura. El tercero explica la creación de una hoja por mayoría, necesaria cuando quedan ejemplos con etiquetas distintas y ya no existen atributos útiles para seguir dividiendo.

Durante estas pruebas se detecta que la información interna del algoritmo necesita una preparación antes de mostrarse. Las listas de etiquetas o atributos, si se presentan tal como las maneja el código, tienen un formato propio de programación y resultan poco naturales en una explicación. Por ello, la ventana transforma esos valores en frases legibles, con enumeraciones y textos adaptados al caso concreto. Esta decisión marca el tono de la aplicación, que gira alrededor de que la interfaz debe traducir las estructuras internas del programa a una explicación comprensible para el usuario.

También se empieza a mostrar el criterio utilizado para elegir la siguiente división. Al principio aparece como una lista textual de valores calculados para cada atributo. Aunque es una solución sencilla, ya conecta el cálculo del algoritmo con una justificación visible dentro de la interfaz. A partir de este punto, la construcción del árbol queda acompañada por una justificación de cada paso.

5.2.3 Animación de la construcción del árbol

Cuando la construcción paso a paso está funcionando, se aprecia otro problema de lectura. Una pulsación puede crear un nodo, desplazar parte del árbol y dibujar varias ramas nuevas. El resultado lógico es correcto, aunque el cambio visual puede ser demasiado brusco para seguirlo con claridad. Por esta razón se incorporan animaciones a la construcción.

Las primeras animaciones se aplican a las conexiones. Cada rama se dibuja progresivamente desde el nodo padre hasta el nodo hijo, de manera que el usuario pueda identificar qué parte acaba de aparecer. Las etiquetas de las ramas también se integran en la conexión: la línea deja un espacio para el texto y este se orienta siguiendo la inclinación de la rama, con



Figura 5.5: Interfaz de entrenamiento del árbol tras traducir los controles. A la izquierda se muestra el botón para comenzar el entrenamiento y a la derecha el estado del árbol al avanzar con el botón de siguiente paso, con temas distintos para nodos internos y hojas.

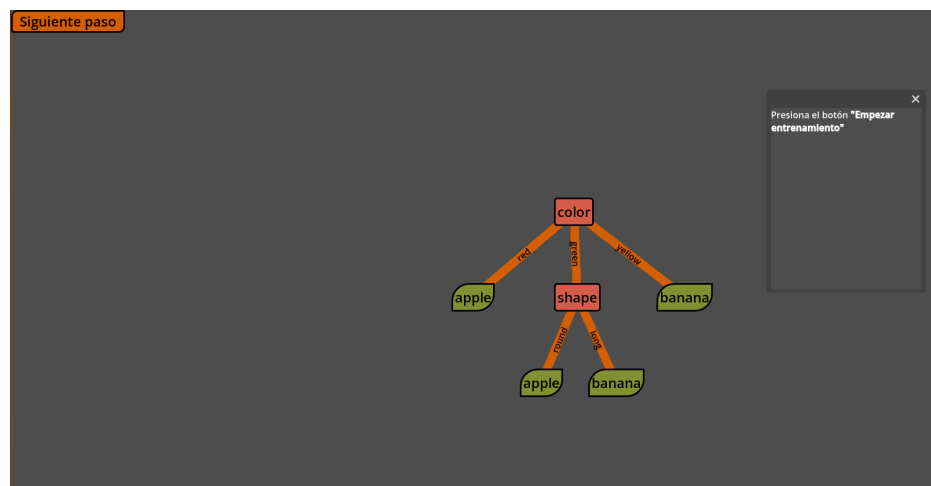


Figura 5.6: Primera versión de la ventana informativa integrada en la vista del árbol, todavía con un mensaje inicial que indica al usuario que debe comenzar el entrenamiento.

la corrección necesaria para que no quede invertido. Así se muestra el valor del atributo que conduce a cada hijo sin llenar los nodos de información secundaria.

Después se anima la aparición de los nodos. En vez de mostrarse de golpe, los nodos nuevos aparecen mediante una transición de opacidad. Este efecto tiene una finalidad funcional, no solo estética, ya que dirige la atención hacia el elemento incorporado en el paso actual y ayuda a distinguirlo de los nodos que ya estaban presentes.

otro caso donde creo que suefa forzado La introducción de animaciones obliga a revisar la actualización de conexiones. En una versión temprana, al recalcular el árbol se redibujaban todas las ramas. Eso provocaba que una conexión antigua volviese a animarse cada vez que se añadía una nueva. La solución consiste en conservar las conexiones existentes, actualizar su posición si el árbol se recoloca y animar únicamente las conexiones creadas en ese paso. Con ello, el movimiento de la escena comunica mejor qué elemento pertenece al avance actual.

En esta misma etapa se añaden los nodos pendientes. Cuando el algoritmo crea un nodo interno, ya conoce los valores del atributo elegido y, por tanto, las ramas que saldrán de él. Todavía queda por decidir si cada rama terminará en una hoja o en otro nodo de decisión. Hasta este momento, esta incertidumbre no era representada de ninguna forma, ya que las ramas no eran dibujadas hasta que el siguiente nodo era definido. Pero como el objetivo es representar la información que tiene el árbol en cada instante, se optó por crear nodos temporales con un texto provisional y un estilo propio que los diferencia. **aquí he metido un pretérito que no se si desentona mucho** Así las conexiones son dibujadas cuándo el algoritmo las tiene en cuenta. Los nodos pendientes muestran que una rama ya está prevista y que su contenido se resolverá en un paso posterior. Cuando el algoritmo procesa esa rama, el nodo temporal se transforma en el nodo definitivo, con su atributo o su etiqueta final. La transformación también se anima: el nodo reduce su opacidad, cambia de texto y tema visual, y vuelve a mostrarse. De esta manera, el usuario percibe que se está completando una parte del árbol que ya existía como posibilidad abierta. **No se si es correcto poner estas "afirmaciones" sobre el usuario** En la Figura 5.7 se puede ver el aspecto de estos nodos pendientes.

También se añade una forma de obtener información de los nodos una vez estos han pasado. Al pasar por ciertos nodos, el texto podía cambiar para mostrar un valor numérico asociado a la decisión. La idea permitía consultar información sin recargar visualmente todo el árbol, aunque un número aislado resultaba difícil de interpretar fuera de contexto. Esta prueba ayuda a confirmar que la explicación principal debía concentrarse en la ventana informativa, donde los valores pueden acompañarse de texto y comparaciones visuales.

5.2.4 Gráficas y casos explicativos en la ventana

La ventana informativa evoluciona a medida que los pasos del árbol se vuelven más complejos. Una explicación en texto plano resulta suficiente para los primeros casos, aunque se

queda corta cuando el algoritmo compara varios atributos y escoge uno de ellos. El objetivo pasa entonces a combinar texto y apoyo visual, de forma que el usuario pueda ver la comparación en lugar de leer solo una lista de números.

El cambio principal es la incorporación de una gráfica de barras, mostrada en la Figura 5.8. La gráfica representa la métrica calculada para cada atributo, basada en la entropía y utilizada por el algoritmo como ganancia de información. Esta representación permite comparar de un vistazo qué atributo reduce mejor la incertidumbre de los datos y por qué se utiliza para abrir las ramas del nodo actual.

Para integrar la gráfica, la ventana se reorganiza como una zona desplazable con varios elementos. El texto se divide en dos partes: primero se presentan los datos que llegan al nodo, las etiquetas y los atributos disponibles; después se inserta la gráfica; al final se explica el atributo elegido y las ramas que se van a crear. Esta estructura evita que la gráfica aparezca como un elemento aislado y la sitúa dentro del relato del paso actual.

El cambio también modifica la forma en la que se preparan los datos explicativos. Los valores de la métrica dejan de generarse únicamente como frases ya formateadas y se conservan como números. La misma información puede usarse entonces para dos fines: construir el texto de la explicación y alimentar la gráfica de barras. Esta separación mantiene la ventana sincronizada con el cálculo real del algoritmo y evita duplicar la lógica de la métrica en la interfaz.

Durante las pruebas aparece un caso importante: varios atributos pueden obtener el mismo valor máximo de la métrica. Si la aplicación escoge uno de ellos sin explicar la situación, la decisión parece arbitraria. Para cubrir este caso se añade una explicación específica de empate. La ventana indica que hay varias alternativas equivalentes y que el algoritmo continúa con una de ellas para poder seguir construyendo el árbol.

También se revisan los textos asociados a las hojas. Las primeras redacciones estaban pensadas para conjuntos muy pequeños, donde a veces solo llegaba un ejemplo a cada resultado final. Al probar datos más variados, se hizo necesario explicar hojas creadas a partir de varios ejemplos con la misma etiqueta y hojas creadas por mayoría. La ventana pasa así a describir el motivo de una clasificación final de forma más general, sin depender del tamaño exacto del subconjunto que llega a la rama.

Este proceso de ajuste se repite durante toda la fase del árbol. Cada nuevo conjunto de datos revela situaciones que la explicación inicial no cubría bien: ramas puras, atributos agotados, empates, varios ejemplos con la misma clasificación o decisiones que crean nodos pendientes. La ventana va incorporando esos casos para que el usuario no tenga que interpretar silencios de la interfaz.

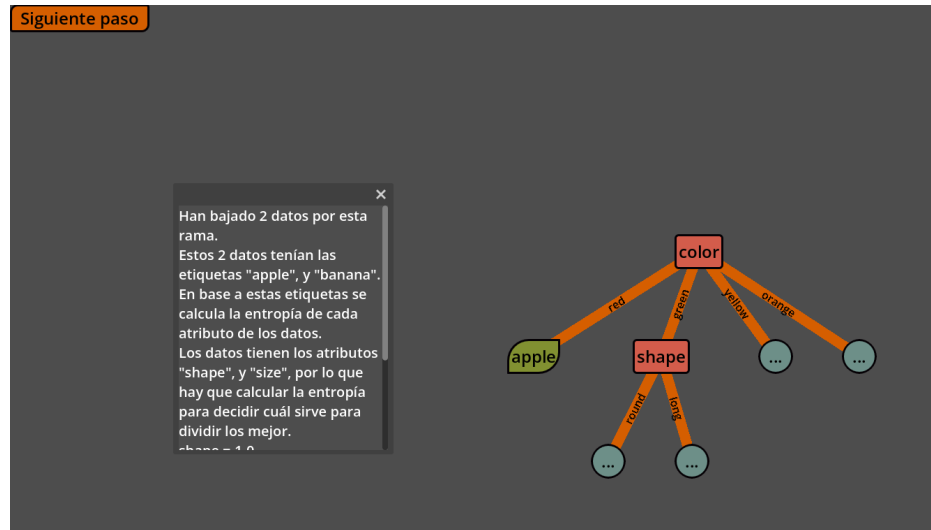


Figura 5.7: Entrenamiento del árbol en el que se aprecian los nodos pendientes.

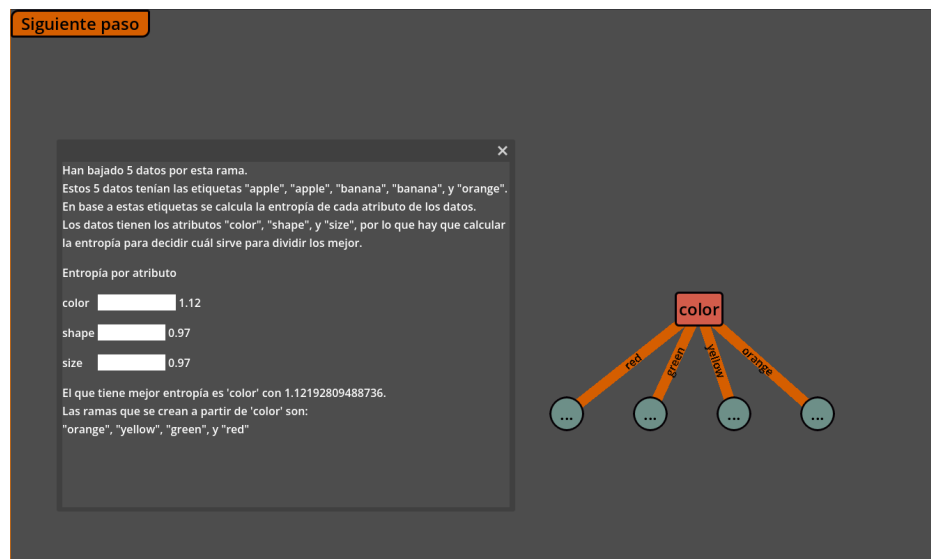


Figura 5.8: Ventana informativa en la que se dibuja la gráfica de la entropía de cada atributo.

5.2.5 Primer panel de selección de datos

Hasta este momento, los datos de entrenamiento están definidos dentro del propio proyecto. Esta decisión facilita las primeras pruebas, porque permite repetir siempre el mismo ejemplo y centrar el esfuerzo en el algoritmo y la visualización. Cuando la construcción del árbol se estabiliza, esa limitación empieza a ser relevante, ya que es muy interesante que una herramienta didáctica permita observar cómo cambian las decisiones al cambiar los datos.

El primer paso para resolverlo es probar un segundo conjunto preparado manualmente. Este nuevo ejemplo incluye atributos y etiquetas distintos, y fuerza situaciones que el primer conjunto apenas mostraba: ramas más profundas, hojas por mayoría y casos con menos atributos disponibles. Gracias a esta prueba se comprueba que el algoritmo no depende de nombres concretos de columnas y que los textos de la ventana siguen siendo válidos con datos diferentes.

A partir de ahí se crea el panel de selección que se aprecia en la Figura 5.9. Su función es guiar al usuario antes de iniciar el entrenamiento. Primero se elige una fuente de datos, después se valida la selección y finalmente se activa el entrenamiento. La primera opción permite usar un conjunto incluido en la aplicación, mientras que la segunda prepara la carga de archivos CSV.

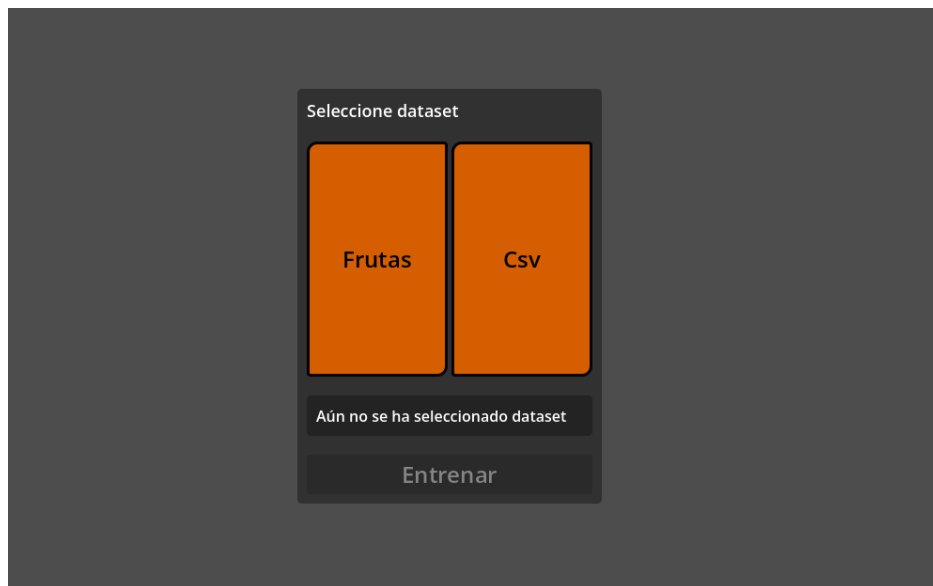


Figura 5.9: Menú que aparece tras seleccionar el modo algorítmico para escoger el conjunto de datos de entrenamiento.

La aparición de este panel cambia el flujo del modo algorítmico. Antes, el usuario comenzaba el entrenamiento directamente. Con el panel, primero selecciona los datos, recibe un mensaje de confirmación o error y solo después puede avanzar por la construcción del árbol.

La vista del árbol y la ventana informativa permanecen ocultas hasta que existe una selección válida, lo que evita mostrar una escena sin contexto.

La comunicación entre el panel de selección, el controlador y la vista se resuelve mediante señales. Cuando el usuario confirma un conjunto de datos, el panel emite la información seleccionada y el controlador inicia el algoritmo con esos datos y atributos. Esta solución encaja con la organización de Godot, ya que permite que componentes situados en partes distintas de la escena colaboren sin depender directamente unos de otros.

La columna objetivo se generaliza en esta etapa. En las primeras pruebas, el algoritmo podía asumir un nombre fijo para la etiqueta de salida. Con datasets distintos, esa suposición deja de ser adecuada. La solución adoptada consiste en tratar como atributos las columnas indicadas por el panel y deducir la etiqueta como la columna restante. De esta forma, el árbol puede entrenarse con conjuntos que usen nombres como *class*, *label*, *y* o una etiqueta en castellano, siempre que el conjunto tenga una única columna objetivo.

5.2.6 Carga de archivos y navegación por pasos

Tras introducir el panel, se completa la carga de CSV para que los archivos seleccionados puedan alimentar el entrenamiento. La lectura se plantea de forma práctica y limitada al tipo de datos que maneja esta versión del árbol: tablas simples, con una primera fila de cabeceras, varias filas de ejemplos y valores categóricos. La aplicación ignora filas vacías, transforma cada fila en un diccionario de valores y separa la columna objetivo del resto de atributos mediante nombres habituales o mediante la diferencia entre columnas de entrada y salida.

El panel informa al usuario del resultado de la carga. Si el archivo puede leerse y la estructura es válida, se activa el botón de entrenamiento. Si aparece un problema, se muestra un mensaje de error y se impide continuar. Esta comprobación es sencilla, aunque suficiente para evitar que el algoritmo empiece con datos incompletos o sin una etiqueta identificable.

Con la selección de datos integrada, el entrenamiento gana una fase previa clara. El usuario escoge datos, confirma la selección, ve aparecer la escena del árbol y utiliza los botones de avance para construirlo. Esta separación hace explícita la dependencia entre el conjunto de entrada y la forma final del árbol.

El siguiente cambio importante es la navegación hacia atrás. Hasta entonces, el entrenamiento solo avanzaba, lo que obligaba a reiniciar el proceso si el usuario quería revisar una decisión anterior. Para una herramienta educativa, poder retroceder es especialmente útil: permite observar de nuevo por qué se abrió una rama, por qué apareció una hoja o qué atributo se escogió en un nodo interno. Por eso se añade el botón “Paso previo”, visible en la Figura 5.10.

El retroceso se apoya en la misma idea que permite avanzar por pasos. El algoritmo mantiene una colección de contextos pendientes y otra de contextos ya ejecutados. Al avanzar, un

contexto pasa de pendiente a ejecutado. Al retroceder, el último contexto ejecutado vuelve a quedar pendiente y la escena debe deshacer su efecto visual. Esta estructura evita reconstruir todo el árbol desde cero cada vez que se pulsa “Paso previo”.

Hacer que el retroceso sea coherente requiere tratar varios casos. Si se deshace un nodo interno, también deben retirarse los contextos hijos y los nodos visuales que dependen de él. Si se deshace una rama que estaba representada por un nodo pendiente, conviene restaurar ese estado provisional en vez de eliminar toda la indicación visual. Si el usuario retrocede hasta el inicio, la raíz desaparece y el árbol debe quedar vacío.

Durante las pruebas se detecta un fallo en ese último caso. Al volver al paso cero, podían quedar conexiones visibles aunque los nodos ya se hubiesen eliminado. La corrección consiste en recalcular también las conexiones cuando el árbol queda sin raíz o sin nodos. Con ello, retroceder hasta el inicio deja la vista en un estado consistente y permite avanzar de nuevo sin restos visuales de la ejecución anterior.

5.2.7 Primera evaluación visual del árbol

El último avance de esta fase es la primera versión del modo de evaluación. Hasta este punto, la aplicación enseña cómo se construye el árbol. Queda por mostrar cómo se usa ese árbol ya entrenado para clasificar nuevos ejemplos. La evaluación se plantea de forma visual: cada dato de prueba se representa como una pieza que se desplaza por la estructura siguiendo las ramas aprendidas.

Para introducir este modo se añaden nuevos controles al panel del algoritmo. Durante el entrenamiento, la evaluación permanece desactivada. Cuando el árbol termina de construirse, el botón de avanzar se bloquea y se habilita la opción de evaluar. Al activarla, se reutiliza el panel de selección de datos para escoger un conjunto de prueba. Así se distinguen dos momentos: primero se seleccionan datos para entrenar y después se seleccionan datos para comprobar el árbol resultante.

La evaluación utiliza una representación sencilla de cada ejemplo, como se aprecia en la Figura 5.11. Cada dato se dibuja como una pieza pequeña que entra en el árbol al pulsar el botón correspondiente. El recorrido se calcula leyendo el atributo de cada nodo interno y buscando la rama cuyo valor coincide con el dato evaluado. Cuando la pieza llega a una hoja, el recorrido termina y queda representada la clasificación obtenida.

Para evitar dependencias innecesarias con el árbol en construcción, la evaluación toma una copia de la información relevante de los nodos visibles: posición, profundidad, atributo, etiqueta, valor de rama y padre. Con esa información puede mover los datos por la escena y decidir el siguiente nodo sin alterar la estructura entrenada. El movimiento entre nodos se anima, de forma que el usuario ve el recorrido como una secuencia de decisiones y no como un salto directo al resultado final.

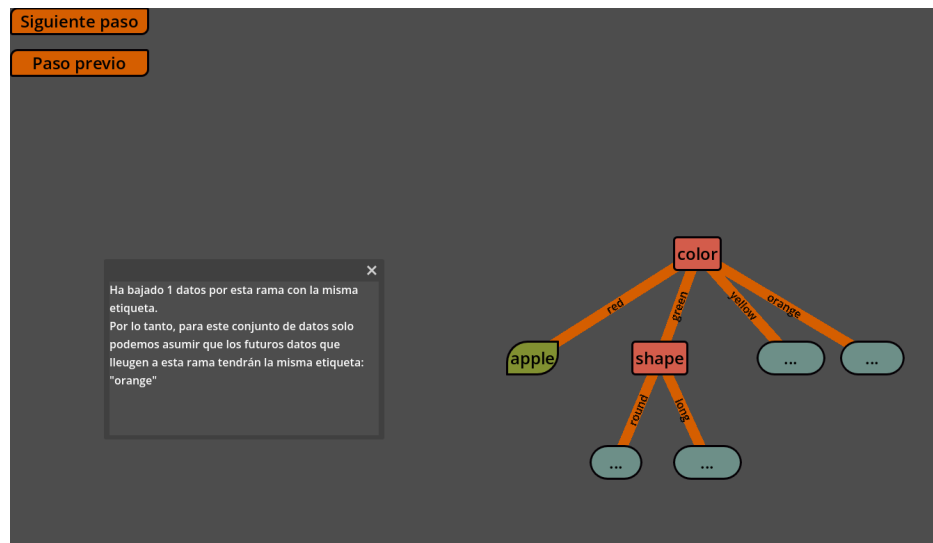


Figura 5.10: Vista del árbol con el botón para retroceder al paso anterior.

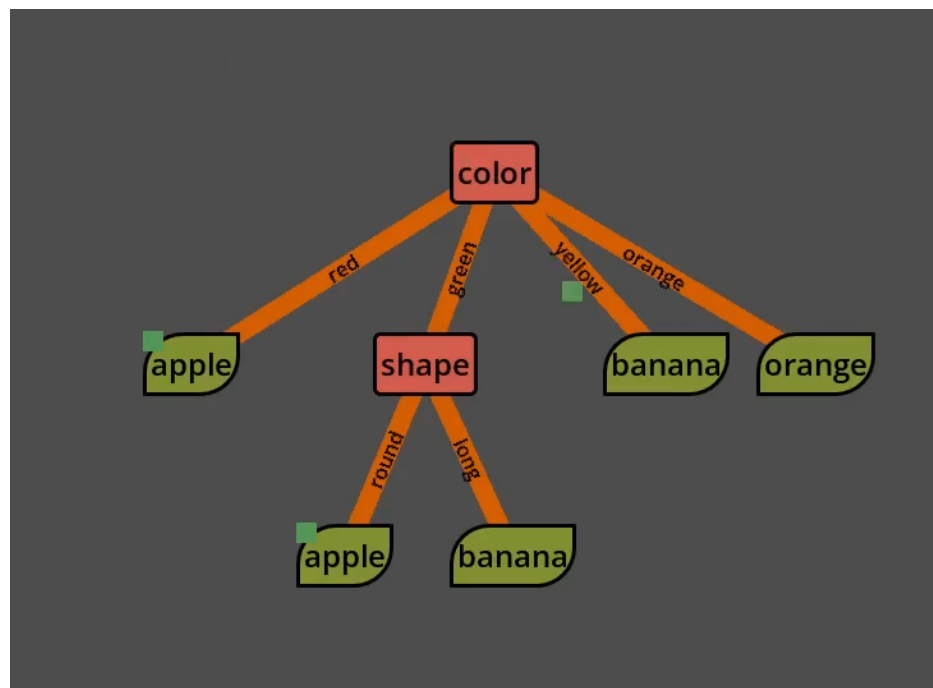


Figura 5.11: Fase de evaluación en la que se ven dos datos ya clasificados y uno en proceso de clasificación.

Esta primera versión del modo de evaluación es todavía básica. Los datos se representan con formas simples y cada pulsación deja caer un ejemplo que recorre el árbol hasta una hoja. Quedan para fases posteriores mejoras como agrupar visualmente los ejemplos bajo las hojas, enriquecer la explicación de cada decisión o introducir una navegación más detallada del recorrido. Aun así, esta prueba establece la idea central del modo de evaluación: mostrar físicamente cómo un ejemplo atraviesa las decisiones aprendidas hasta llegar a una clasificación.

Con este conjunto de cambios queda completada la primera versión significativa del árbol de decisión. El proyecto pasa de dibujar elementos conectados a entrenar un árbol categórico paso a paso, mantener una estructura lógica sincronizada con la escena, mostrar ramas con valores, distinguir nodos internos, hojas y nodos pendientes, animar los cambios, explicar las decisiones con texto y gráficas, cargar datos sencillos, retroceder por la construcción y evaluar ejemplos nuevos de forma visual. A partir de aquí, el desarrollo continúa con el segundo algoritmo de la aplicación: la red neuronal.

5.3 Red neuronal: versión 0.3

5.3.1 Primera arquitectura visual de la red

5.3.2 Configuración de la arquitectura y representación lógica

5.3.3 Selección de datos y restricciones de la red

5.3.4 Algoritmo de entrenamiento incremental

5.3.5 Forward pass y visualización de datos

5.3.6 Cálculo del error y backward pass

5.3.7 Ventana informativa de la red neuronal

5.3.8 Inferencia con la red entrenada

Conclusiones

DERRADEIRO capítulo da memoria, onde se presentará a situación final do traballo, as leccións aprendidas, a relación coas competencias da titulación en xeral e a mención en particular, posibles liñas futuras,...

Capítulo 7

Trabajo futuro

Apéndices

Material adicional

EXEMPLO de capítulo con formato de apéndice, onde se pode incluír material adicional que non teña cabida no corpo principal do documento, suxeito á limitación de 80 páxinas establecida no regulamento de TFGs.

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Etiam lobortis facilisis sem. Nullam nec mi et neque pharetra sollicitudin. Praesent imperdiet mi nec ante. Donec ullamcorper, felis non sodales commodo, lectus velit ultrices augue, a dignissim nibh lectus placerat pede. Vivamus nunc nunc, molestie ut, ultricies vel, semper in, velit. Ut porttitor. Praesent in sapien. Lorem ipsum dolor sit amet, consectetur adipiscing elit. Duis fringilla tristique neque. Sed interdum libero ut metus. Pellentesque placerat. Nam rutrum augue a leo. Morbi sed elit sit amet ante lobortis sollicitudin. Praesent blandit blandit mauris. Praesent lectus tellus, aliquet aliquam, luctus a, egestas a, turpis. Mauris lacinia lorem sit amet ipsum. Nunc quis urna dictum turpis accumsan semper.

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Etiam lobortis facilisis sem. Nullam nec mi et neque pharetra sollicitudin. Praesent imperdiet mi nec ante. Donec ullamcorper, felis non sodales commodo, lectus velit ultrices augue, a dignissim nibh lectus placerat pede. Vivamus nunc nunc, molestie ut, ultricies vel, semper in, velit. Ut porttitor. Praesent in sapien. Lorem ipsum dolor sit amet, consectetur adipiscing elit. Duis fringilla tristique neque. Sed interdum libero ut metus. Pellentesque placerat. Nam rutrum augue a leo. Morbi sed elit sit amet ante lobortis sollicitudin. Praesent blandit blandit mauris. Praesent lectus tellus, aliquet aliquam, luctus a, egestas a, turpis. Mauris lacinia lorem sit amet ipsum. Nunc quis urna dictum turpis accumsan semper.

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Etiam lobortis facilisis sem. Nullam nec mi et neque pharetra sollicitudin. Praesent imperdiet mi nec ante. Donec ullamcorper, felis non sodales commodo, lectus velit ultrices augue, a dignissim nibh lectus placerat pede. Vivamus nunc nunc, molestie ut, ultricies vel, semper in, velit. Ut porttitor. Praesent in sapien. Lorem ipsum dolor sit amet, consectetur adipiscing elit. Duis fringilla tristique neque.

Sed interdum libero ut metus. Pellentesque placerat. Nam rutrum augue a leo. Morbi sed elit sit amet ante lobortis sollicitudin. Praesent blandit blandit mauris. Praesent lectus tellus, aliquet aliquam, luctus a, egestas a, turpis. Mauris lacinia lorem sit amet ipsum. Nunc quis urna dictum turpis accumsan semper.

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Etiam lobortis facilisis sem. Nullam nec mi et neque pharetra sollicitudin. Praesent imperdiet mi nec ante. Donec ullamcorper, felis non sodales commodo, lectus velit ultrices augue, a dignissim nibh lectus placerat pede. Vivamus nunc nunc, molestie ut, ultricies vel, semper in, velit. Ut porttitor. Praesent in sapien. Lorem ipsum dolor sit amet, consectetur adipiscing elit. Duis fringilla tristique neque. Sed interdum libero ut metus. Pellentesque placerat. Nam rutrum augue a leo. Morbi sed elit sit amet ante lobortis sollicitudin. Praesent blandit blandit mauris. Praesent lectus tellus, aliquet aliquam, luctus a, egestas a, turpis. Mauris lacinia lorem sit amet ipsum. Nunc quis urna dictum turpis accumsan semper.

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Etiam lobortis facilisis sem. Nullam nec mi et neque pharetra sollicitudin. Praesent imperdiet mi nec ante. Donec ullamcorper, felis non sodales commodo, lectus velit ultrices augue, a dignissim nibh lectus placerat pede. Vivamus nunc nunc, molestie ut, ultricies vel, semper in, velit. Ut porttitor. Praesent in sapien. Lorem ipsum dolor sit amet, consectetur adipiscing elit. Duis fringilla tristique neque. Sed interdum libero ut metus. Pellentesque placerat. Nam rutrum augue a leo. Morbi sed elit sit amet ante lobortis sollicitudin. Praesent blandit blandit mauris. Praesent lectus tellus, aliquet aliquam, luctus a, egestas a, turpis. Mauris lacinia lorem sit amet ipsum. Nunc quis urna dictum turpis accumsan semper.

Bibliografía

- [1] G. Sanderson, “Neural networks,” 3Blue1Brown, consultado el 11 de junio de 2026. [En línea]. Disponible en: https://www.youtube.com/playlist?list=PLZHQObOWTQDNU6R1_67000Dx_ZCJB-3pi
- [2] —, “But what is a neural network?” 3Blue1Brown, consultado el 11 de junio de 2026. [En línea]. Disponible en: <https://www.3blue1brown.com/?topic=neural-networks>
- [3] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, “Gradient-based learning applied to document recognition,” *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998.
- [4] D. Smilkov and S. Carter, “A neural network playground,” TensorFlow, consultado el 11 de junio de 2026. [En línea]. Disponible en: <https://playground.tensorflow.org/>
- [5] M. Kahng, N. Thorat, P. Chau, F. Viégas, and M. Wattenberg, “Gan lab: Play with generative adversarial networks in your browser!” Georgia Tech and Google Brain/PAIR, consultado el 11 de junio de 2026. [En línea]. Disponible en: <https://poloclub.github.io/ganlab/>
- [6] Interactive ML, “Interactive decision trees learning tool,” consultado el 11 de junio de 2026. [En línea]. Disponible en: <https://www.interactive-ml.com/decision-tree.html>
- [7] Godot Engine contributors, “Godot engine,” consultado el 11 de junio de 2026. [En línea]. Disponible en: <https://godotengine.org/>
- [8] —, “Godot documentation,” consultado el 11 de junio de 2026. [En línea]. Disponible en: <https://docs.godotengine.org/>
- [9] —, “Exporting for the web,” consultado el 11 de junio de 2026. [En línea]. Disponible en: https://docs.godotengine.org/en/stable/tutorials/export/exporting_for_web.html
- [10] S. Chacon and B. Straub, *Pro Git*, 2nd ed. Apress, 2014, consultado el 11 de junio de 2026. [En línea]. Disponible en: <https://git-scm.com/book/en/v2>

- [11] GitHub, Inc., “Github pages documentation,” consultado el 11 de junio de 2026. [En línea]. Disponible en: <https://docs.github.com/en/pages>